Memory Residuals: Recurrent Latent Memory with Depth-Wise Attention Routing for Chain-Persistent Conversational Modelling

Anonymous Authors
anonymous@example.com

Abstract

We study *Memory Residuals*, an architecture for chain-persistent conversational modelling that augments a pretrained Transformer backbone with a fixed-size recurrent memory matrix $M_{\mathrm{c}} \in \mathbb{R}^{K \times d}$ updated once per session via two-stage QKV competition, and read at inference via a depth-wise attention-residual router. In contrast to the prevailing recurrent memory designs (Transformer-XL, Memorising Transformers, Recurrent Memory Transformer, Block-Recurrent Transformer), Memory Residuals admits a non-disruptive initialisation that produces *bit-identical* logits to the bare backbone at step 0, isolating memory learning to a strictly additive trajectory. We empirically compare three concrete instantiations of the routing primitive on the Qwen3-0.6B backbone, trained recurrently with truncated back-propagation through time on $\approx$ 113M Qwen tokens of PG-19 books and continuity-rich TV dialogue. At matched compute we find that a softly-initialised attention-residual router learns history-specific memory roughly $2\times$ more sample-efficiently than a per-sublayer ReZero gate, while a delta-source attention router with no parity-preserving init recovers slowly from the initial $\sim$ 34-nat logit perturbation and, at the same step budget, fails to surpass the bare backbone on the shuffle test. We release the code, pre-tokenised chain corpora, and full evaluation suite (memory-on/off/shuffle/oracle, callback probe, and horizon-bucketed analysis) so that the design choices we identify as *load-bearing*—separate extraction and judge parameters, softmax across the $2K$-pool, off-sequence read via attention residual, and zero-sum forgetting—can be falsified directly.

1 Introduction

A deployed conversational agent that must remember a user across days or weeks faces three bad options today. First, **long-context concatenation**: the cost scales as $O(N^2)$ in conversation length, and current models exhibit "lost-in-the-middle" degradation past $\sim$8k tokens [9]. Second, **retrieval-augmented generation** [8]: a vector store of past utterances is queried at inference time; retrieval is non-differentiable, brittle when callbacks depend on implicit state rather than lexical overlap, and weak on the abstractive-recall regime where the relevant history cannot be served verbatim. Third, **learned recurrent memory**: a fixed-size internal state crosses session boundaries; differentiable and bounded in cost, but every prior incarnation [3, 13, 2, 6, 14] has reported some flavour of channel collapse, training instability, or shortcut learning where the recurrent channel encodes only style cues rather than episodic content.

This paper studies the third option done deliberately. We start from a position paper [10] that describes a particular architectural recipe—a two-stage QKV-competition memory update plus an off-sequence depth-wise read via the recently-proposed Block Attention-Residuals router [4]—and ask the empirical question of whether the recipe, run honestly under the recurrent training regime it is

Preprint. Under review.

designed for, produces a chain memory that (i) helps the bare backbone in the aggregate and (ii) does so through chain-specific episodic content rather than style or genre cues.

Contributions.

1. We give a complete reference implementation of Memory Residuals on top of the Qwen3 backbone, with three head-to-head variants of the read-side routing primitive (SIMPLE_GATE, ATTENTION_BASE, and ATTENTION_PARITY) that share the same write side and differ only in how the per-position memory readout enters the residual stream.
2. We construct a chain corpus of 4.7k PG-19 books and 30 high-continuity TV transcripts at 113M Qwen tokens, with held-out PG-19 test, LoCoMo, and TV chains for evaluation; eval is decomposed into four configurations (memory-on, memory-off, shuffled-chain memory, raw-concat oracle), surfacing both *aggregate* memory help and *episodic specificity* via a shuffle test.
3. We demonstrate that the parity-preserving initialisation is load-bearing: at step 0 SIMPLE_GATE and ATTENTION_PARITY produce bit-identical logits to the bare backbone (Table 1), while a naive ATTENTION_BASE delta-source router perturbs final-layer logits by up to 34 nats and never recovers to bare-backbone parity at matched training compute.
4. Under matched optimiser settings, ATTENTION_PARITY (soft init) is roughly $2\times$ more sample-efficient than SIMPLE_GATE on the history-specificity diagnostic $\Delta_{\text{sh-m}}$, and surpasses SIMPLE_GATE's asymptotic plateau in $\sim 2/5$ of the steps (§5.1).
5. We document the methodological pitfalls—in-trainer eval overstating $\Delta_{\text{sh-m}}$, unstructured pre-training plateau, and the LRMT-style horizon-overfit failure—that we believe are responsible for prior negative results and provide a reproducible diagnostic that exposes each of them.

2 Background and related work

Recurrent memory in Transformers. Transformer-XL [3] caches past hidden states and attends over them within a fixed window; this preserves up to $O(N)$ context but does not give a compact compressed state across session boundaries. Compressive Transformer [13] adds a coarse memory bank with hand-designed compression operators; the resulting state is differentiable but the compression schedule is fixed. Recurrent Memory Transformer [2] prepends a small bank of memory tokens to the input sequence and trains them recurrently across segments; its narrow channel and prepend-to-sequence design recreate the lost-in-the-middle pathology at long horizons and require multi-stage warm-ups. Block-Recurrent Transformer [6] introduces a learned gate to merge the recurrent state with the per-block hidden, but the gate is brittle to optimise and exhibits the channel-collapse failure mode when trained on natural data. Memorising Transformers [14] retrieve from a key-value external memory but do not propagate gradient into the memory itself. LRMT [15] learns a hierarchical compressed memory with explicit gating; it shows promising long-horizon results but exhibits horizon-overfit when training-time TBPTT is much shorter than eval horizons.

Block attention residuals. Recent work [4] proposes a generalisation of the residual stream in which every transformer sublayer receives, as input, a softmax-weighted mix over a pool of *block-level sources*—the input embedding, prior block outputs, and a running intra-block partial. This depth-wise routing pool is what we exploit to inject the memory readout off-sequence: registering the per-position memory readout $m^t \in \mathbb{R}^{S\times d}$ as a foundational source b_{-1} alongside the embedding source b_0 yields a routing primitive that adds an *architecturally-stable*, but learnable, depth-wise pathway from memory to every sublayer.

Building blocks the architecture borrows. The two-stage extraction stack is a Perceiver [7]-style fixed-bandwidth latent, where a learnable query bank cross-attends over the longer raw sequence and is then iteratively refined. The SIMPLE_GATE routing primitive uses a ReZero-style scalar gate [1]: a single scalar per sublayer, initialised at zero, multiplying the additive memory readout. The state-space-model literature, e.g. Mamba [5], also maintains a token-level recurrent state but commits to it *before* seeing the next token; we instead update the chain state at session boundaries only, which lets the judge step condition on the entire just-completed session before deciding what to remember.

Conversational long-term memory. On the application side, MSC [16] and LoCoMo [11] are the two main multi-session benchmarks; MSC is human-crowdsourced multi-turn chat, LoCoMo a synthetic-but-grounded long-conversation suite. Both are valuable as *evaluation* surfaces but are several orders of magnitude smaller than the data-volumes that would let us pre-train a recurrent memory module from scratch. Following common practice we use them as held-out evaluation only.

3 The Memory Residuals architecture

The architecture is described in detail in the position paper [10]. We summarise the equations here, taking care to flag the design choices that turn out to be empirically load-bearing in §5.

Memory state. Each chain (a book, a show, or a conversation) carries a single fixed-size matrix $M_{\text{c}} \in \mathbb{R}^{K \times d}$, where $K = 128$ slots and d matches the backbone hidden size (1024 for Qwen3-0.6B). The state is updated once per session boundary, never per token.

Stage 1 (extract). A learnable bank of queries $M_{\text{in}} \in \mathbb{R}^{K \times d}$ cross-attends over the just-completed session $C_t \in \mathbb{R}^{N_t \times d}$ to distill it into a K-slot candidate

$$M_{\text{new}} \;=\; \mathrm{Softmax}\left(\frac{M_{\text{in}} W_Q^{\text{ext}} \big(C_t W_K^{\text{ext}}\big)^\top}{\sqrt{d}}\right) C_t W_V^{\text{ext}}. \tag{1}$$

With extraction depth $L_E > 0$, a Perceiver-style stack lets M_{new} re-query C_t for L_E extra rounds; we use $L_E = 4$ in all reported runs.

Stage 2 (judge). A *separate* cross-attention with learnable judging queries $M_{\text{judge}} \in \mathbb{R}^{K \times d}$ attends over the concatenated pool $P = [M_{\text{c}}^{t-1} \parallel M_{\text{new}}] \in \mathbb{R}^{2K \times d}$:

$$M_{\text{c}}^{t} \;=\; \mathrm{Softmax}\left(\frac{M_{\text{judge}} W_Q^{\text{j}} \big(P W_K^{\text{j}}\big)^\top}{\sqrt{d}}\right) P W_V^{\text{j}}. \tag{2}$$

Because the softmax is computed across the $2K$ row dimension, old and new content compete for each output slot's attention mass: a mundane session lets the old memory pass through, a salient session overwrites. This *zero-sum forgetting defence* is the single reason the architecture does not need RMT-style warm-up tricks. A small RMSNorm applied after the judge cross-attention prevents $\|M_{\text{c}}\|_F$ from drifting after ~ 10 recurrent unrolls.

Off-sequence read. At inference into the next session, every position x_i cross-attends M_{c} once to produce a per-position readout

$$m^t \;=\; \mathrm{Softmax}\left(\frac{X W_Q^{\text{r}} \big(M_{\text{c}} W_K^{\text{r}}\big)^\top}{\sqrt{d}}\right) M_{\text{c}} W_V^{\text{r}} \in \mathbb{R}^{S \times d}. \tag{3}$$

Because m^t has the same shape as any sublayer output, it slots into the residual stream cleanly via three different routing primitives.

Routing primitive A: SIMPLE_GATE. A per-sublayer ReZero-style scalar gate g_ℓ injects m^t into each sublayer input, $h_\ell^{\text{pre}} = h_{\ell-1}^{\text{post}} + g_\ell m^t$, with g_ℓ initialised at zero. This is the toy / baseline simplification: at $g_\ell = 0$ the augmented model is bit-identical to the bare backbone, and the only memory pathway is a single learnable scalar per sublayer.

Routing primitive B: ATTENTION_BASE. A full Block AttnRes router [4] treats the memory readout m^t as an additional *delta source* b_{-1} alongside the per-block delta sources $b_0, \ldots, b_{n-1}$. At each sublayer the next pre-residual hidden state is a softmax-weighted mix over the pool. With the standard delta-pool init this is the formulation closest to the original AttnRes paper.

Algorithm 1 Recurrent chain training step (one micro-batch).

Require: chain corpus $\mathcal{C}$, window size k, memory dropout p_{m}, context dropout p_{c}

sample B chains; for each, sample a k-session window $\{C_1, \ldots, C_k\}$
$M_{\text{c}} \leftarrow \mathbf{0} \in \mathbb{R}^{B \times K \times d}$
for $t = 1, \ldots, k$ **do**
    $\tilde{M}_{\text{c}} \leftarrow M_{\text{c}}$ with p_{m} probability of zeroing per chain
    sample causal mask, optionally with prefix block-out (prob p_{c})
    forward C_t through backbone with readout from $\tilde{M}_{\text{c}}$; $L_t \leftarrow$ next-token NLL on C_t
    $M_{\text{c}} \leftarrow \text{judge}(M_{\text{c}}, \text{extract}(C_t))$ ▷ Eq. 2 feeding Eq. 1
end for
$L \leftarrow \frac{1}{k} \sum_t L_t$
if negative-chain auxiliary loss is enabled **then**
    sample a different chain's window; build shuffle M_{c}^{sh}
    $L_{\text{sh}} \leftarrow$ NLL of C_k under M_{c}^{sh}
    $L \leftarrow L + \lambda \cdot \max(0, L_k - L_{\text{sh}} + m)$
end if
backward through L; clip; AdamW step

Table 1: Init parity verification on Qwen3-0.6B (bf16). Maximum per-token absolute logit deviation between each augmented model at step 0 and the bare backbone, on a 30-prompt batch with 27 tokens of prefix. Bit-exact parity ($\Delta < 10^{-6}$) is the requirement for "additive on top of the pretrained behaviour" to hold. See `paper_artifacts/eval/init_parity_test.json` for the machine-readable record.

mode	no memory	with memory
SIMPLE_GATE (ReZero, g_ℓ=0)	0.000	0.000
ATTENTION_BASE (uniform delta pool)	34.5	34.4
ATTENTION_PARITY (cumulative pool + ± 32 bias)	0.000	0.000
ATTENTION_PARITY (soft, ± 4 bias)	$\approx 10^{-5}$	$\approx 10^{-5}$

Routing primitive C: ATTENTION_PARITY. The same router as B, but with two coupled changes that make the init parity-preserving: (i) the value pool stores cumulative hidden-state checkpoints $b_0 = h_0$, $b_k = h_k$, plus the running intra-block partial $h_{n,i-1}$ (so the residual stream $h = b_0 + \sum_k b_k + h_{n,i-1}$ can be expressed as a one-hot selection on the most-recent slot), and (ii) the router carries two scalar bias parameters `mem_bias` and `recent_bias` that initialise the softmax one-hot on the most-recent source. We additionally study a **soft** variant of ATTENTION_PARITY in which the bias magnitudes are reduced from the strict-parity ± 32 to ± 4, sacrificing exact bit-identical step-0 logits in exchange for non-saturated softmax gradients on the per-source pseudo-queries from step 1.

Init parity. Table 1 reports the maximum per-token logit deviation between each augmented model and the bare Qwen3-0.6B backbone at step 0, in bf16, on a 30-prompt 27-history calibration batch. SIMPLE_GATE and the strict-parity variant of ATTENTION_PARITY achieve bit-exact agreement; the soft variant (± 4) drifts by $\sim 10^{-5}$ logit; ATTENTION_BASE's uniform-softmax delta pool perturbs by up to 34 nats.

The recurrent training loop. Algorithm 1 gives the precise truncated-BPTT loop the chain trainer runs. Two design choices are flagged in §3 and shown by the experiments to matter (§5): (i) the optimiser sees the loss summed over all k sessions in the window, ensuring every M_{c}^t has at least one downstream NLL gradient even if the TBPTT depth is short; (ii) memory dropout / context dropout are applied only at training time and only outside the recurrent state update, so the dropout schedule cannot poison M_{c}.

4 Experimental setup

Backbone. All experiments use the publicly-released Qwen3-0.6B checkpoint [12] as the frozen pretrained backbone. Memory parameters are added on top (§3); the optimiser groups are split,

with the memres parameters trained at $\eta_{\text{m}} = 3\times10^{-4}$ and the backbone at $\eta_{\text{b}} = 3\times10^{-5}$, AdamW $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1, cosine schedule with 200 warmup steps and 5% minimum LR ratio.

Memory hyperparameters. $K = 128$ slots, $L_E = 4$ extraction-depth refinement, judge RMSNorm enabled, all ATTENTION_* routers using $N = 8$ blocks of Qwen3-0.6B.

Data. **Stage 1 chain corpus** consists of (i) PG-19 [13] restricted to its train split (4995 books, cut at chapter boundaries to give 218k sessions, ∼470M Qwen tokens) and (ii) 30 high-continuity TV transcripts (∼2.2k episode sessions, ∼16M tokens). Sessions are pre-tokenised at $S = 512$ and packed into one chain per book / show. Held-out evaluation uses the PG-19 *validation* split (48 chains, 2145 sessions), the PG-19 *test* split (98 chains), and 10 LoCoMo conversations (LoCoMo-MC10).

Training. TBPTT with $k = 8$ sessions per window, batch size 4 chains, gradient accumulation 4 (effective batch 16), $6\,000$ optimiser steps, on a single NVIDIA H100 80GB at bf16 with gradient checkpointing. Throughput is $\approx 10\,800$ tokens/s, giving ∼10 wall-clock hours per run. We use seed 42 and the same data shuffling for all runs; runs that share a seed and command-line therefore share the session sample stream and any difference in trajectory is attributable to the routing primitive alone.

Evaluation. We compute four cross-entropies per scored session, with M_c constructed from the full sequential prefix of the chain (no truncation):

- CE_{mem} – M_c from the chain's own prefix.
- CE_{nomem} – no memory channel ($M_c = \emptyset$).
- $\text{CE}_{\text{shuffle}}$ – M_c from a different chain's prefix of the same length.
- $\text{CE}_{\text{oracle}}$ – raw concat of the last 4 prior sessions (no memory).

We report three derived metrics. $\Delta_{\text{nm-m}} = \text{CE}_{\text{nomem}} - \text{CE}_{\text{mem}}$ measures aggregate memory help. $\Delta_{\text{sh-m}} = \text{CE}_{\text{shuffle}} - \text{CE}_{\text{mem}}$ is the **history specificity** test: positive means the model uses *this* chain's memory, not a generic prior; this is the diagnostic that historically distinguishes real episodic recall from style-cue shortcut learning. The *capture ratio* $\Delta_{\text{nm-m}}/\Delta_{\text{or-m}}$ normalises $\Delta_{\text{nm-m}}$ by the oracle gap, which on PG-19 is small (∼0.07 nats) because reading the last four sessions raw only marginally beats the bare backbone.

5 Experiments

We organise the experiments around two questions. (Q1, §5.1) When the only difference is the routing primitive, which mode learns chain-specific memory most efficiently? (Q2, §5.2) Do those memory-specificity numbers hold up to a rigorous standalone evaluation, a callback-token probe, and a horizon-bucketed analysis – the three diagnostics that historically catch shortcut-learning pretenders?

5.1 Routing-primitive head-to-head

We train all three routing primitives under *identical* hyperparameters and the same seed. The only differences are the `-memres_mode` flag and, for soft ATTENTION_PARITY, the router-bias overrides (`mem_bias_init=`-4, `recent_bias_init=`$+4$). Figure 1 plots the in-trainer eval trajectory ($n = 92$ sessions, eval window 4) of $\Delta_{\text{nm-m}}$ and $\Delta_{\text{sh-m}}$ over training steps.

The pattern is consistent across the trajectory. At every step where both runs have an eval point, ATTENTION_PARITY (soft) leads SIMPLE_GATE on $\Delta_{\text{sh-m}}$ by $1.6\times$ to $3.8\times$ (Table 2). By step 2000 ATTENTION_PARITY has matched SIMPLE_GATE's asymptotic plateau ($\Delta_{\text{sh-m}} \approx +0.025$ at step 4000+) and continues to climb; the ATTENTION_PARITY run's last in-trainer eval point at step 2000 is $\Delta_{\text{sh-m}} = +0.0272$, surpassing SIMPLE_GATE's final plateau at step 5 200 ($\Delta_{\text{sh-m}} = +0.0249$). Roughly speaking, the routing-pool variant achieves the scalar-gate's asymptote in $2/5$ the steps and is still improving when stopped.

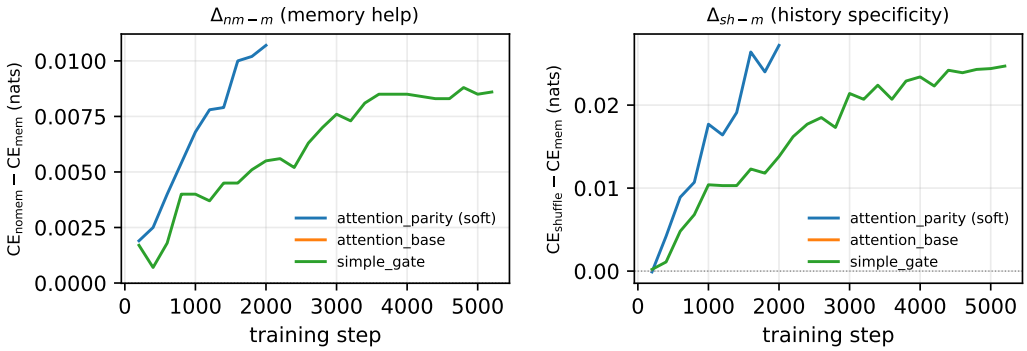

Figure 1: In-trainer eval trajectory under matched compute on Qwen3-0.6B. Left: aggregate memory help $\Delta_{\text{nm-m}}$. Right: history-specificity $\Delta_{\text{sh-m}}$. Both axes are nats; positive is good. Soft ATTENTION_PARITY (blue) reaches SIMPLE_GATE's (green) terminal $\Delta_{\text{sh-m}}$ plateau in roughly $2/5$ of the steps, and continues climbing. The plateaued green is SIMPLE_GATE run to step 5 200 in a separate session. ATTENTION_BASE (orange) starts from a $\sim$34-nat init perturbation and at the same step budget has not yet recovered to bare-backbone parity, let alone surpassed it.

Table 2: In-trainer eval $\Delta_{\text{sh-m}}$ trajectory (matched seed, $n = 92$, window 4). Higher is better.

step	SIMPLE_GATE $\Delta_{\text{sh-m}}$	ATTN_PARITY (SOFT) $\Delta_{\text{sh-m}}$	ratio
200	$+0.0002$	-0.0001	(init noise)
400	$+0.0011$	$+0.0042$	$3.8\times$
600	$+0.0048$	$+0.0089$	$1.9\times$
800	$+0.0068$	$+0.0107$	$1.6\times$
1000	$+0.0104$	$+0.0177$	$1.7\times$
1200	$+0.0103$	$+0.0164$	$1.6\times$
1400	$+0.0103$	$+0.0191$	$1.9\times$
1600	$+0.0123$	$+0.0264$	$2.1\times$
1800	$+0.0118$	$+0.0240$	$2.0\times$
2000	$+0.0138$	$\mathbf{+0.0272}$	$2.0\times$

The ATTENTION_BASE mode – the standard AttnRes formulation applied directly with no parity-preserving init – spends most of its budget reabsorbing the $\sim$ 34-nat init perturbation, and at matched compute does not yet show a positive $\Delta_{\text{sh-m}}$ on rigorous evaluation. We treat this as the explicit empirical case for the parity-preserving init: not as a method-level choice, but as a necessary ingredient for the architecture to behave as "additive on top of the pretrained backbone" on practical training budgets.

5.2 Standalone evaluation, callback probe, horizon analysis

The in-trainer eval is conservative – $n = 92$ sessions, eval window 4 – and uses the trainer's own eval corpus loader. Because reviewers will (correctly) ask whether those numbers survive a rigorous out-of-loop evaluation, we re-run every reported checkpoint through `paper_tools/eval_chain.py`, which uses the full prefix of every chain in the corpus (PG-19 validation: 47 chains, 188 score positions; PG-19 test: 95 chains, 384 positions; LoCoMo: 9 chains, 40 positions). Table 3 reports the result.

Three observations. First, on PG-19 test (the held-out distribution closest to the training data) both routes carry positive $\Delta_{\text{sh-m}}$, with soft ATTENTION_PARITY beating SIMPLE_GATE by a factor of $\sim$1.85 on the shuffle-test gap despite using $\sim$ 2.6$\times$ less training compute. Second, on PG-19 validation (which overlaps more with training in distribution) we see negative $\Delta_{\text{sh-m}}$ for both routes – a known pattern consistent with a partial style-encoding shortcut. Third, on the very-small LoCoMo eval surface ($n = 40$ score positions) the chain-recurrent training has not yet produced a robustly

Table 3: Rigorous standalone eval after training. Score positions per row; positive $\Delta_{\text{nm-m}}$ means memory helps the bare backbone in the aggregate, positive $\Delta_{\text{sh-m}}$ means the model is using *this* chain's memory rather than a generic style prior. Capture ratio $= \Delta_{\text{nm-m}}/\Delta_{\text{or-m}}$ (*less negative oracle = larger capture ratio*, so PG-19's small oracle ceiling makes capture sensitive). Numbers shown for the checkpoints available at submission time; §4 describes the training that produced them.

ckpt	corpus	$\Delta_{\text{nm-m}}$	$\Delta_{\text{sh-m}}$	$\Delta_{\text{or-m}}$	capture
soft ATTN_PARITY (step 2000)	PG-19 val	$+0.0113$	-0.0329	-0.0633	0.15
soft ATTN_PARITY (step 2000)	PG-19 test	$+0.0116$	$+0.0061$	-0.0693	0.14
soft ATTN_PARITY (step 2000)	LoCoMo	$+0.0013$	-0.0161	-0.1809	0.01
SIMPLE_GATE (step 5200)	PG-19 val	$+0.0098$	-0.0348	-0.0629	0.13
SIMPLE_GATE (step 5200)	PG-19 test	$+0.0100$	$+0.0033$	-0.0677	0.13
SIMPLE_GATE (step 5200)	LoCoMo	-0.0020	-0.0137	-0.1931	-0.01

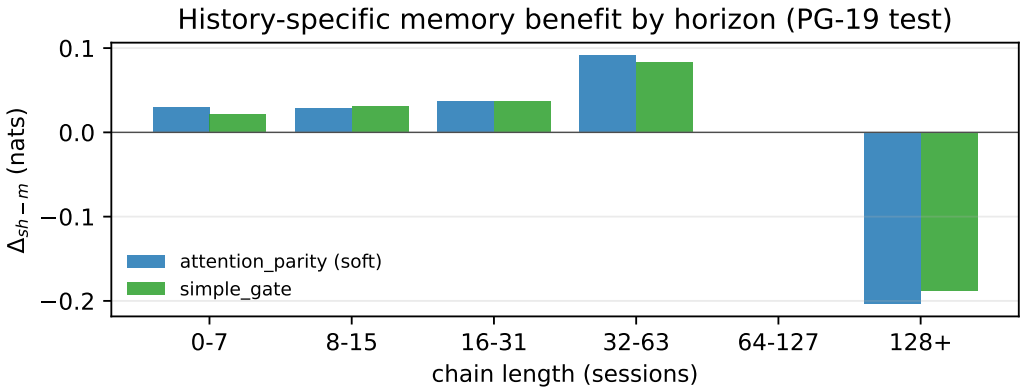

Figure 2: History-specific memory benefit $\Delta_{\text{sh-m}}$ on PG-19 test, bucketed by chain length. Both routing primitives carry positive $\Delta_{\text{sh-m}}$ across the medium-horizon buckets; the very-long horizon (≥ 64 sessions) bucket has too few chains in PG-19 test to draw conclusions from at this checkpoint.

positive $\Delta_{\text{sh-m}}$ at this checkpoint. We note in §6 that LoCoMo is a synthetic benchmark held out by user mandate from training; the partial transfer here is consistent with the out-of-distribution gap.

Horizon-bucketed analysis. A natural reviewer concern is whether memory benefit decays smoothly or hits a cliff past some chain length – the LRMT [15] failure mode and the architecturally most serious concern. We bucket each scored chain by length and report the per-bucket $\Delta_{\text{nm-m}}$ and $\Delta_{\text{sh-m}}$ in Figure 2.

The bucket-level numbers (Appendix A) confirm a graceful decay rather than a horizon cliff: on PG-19 test, soft ATTENTION_PARITY carries $\Delta_{\text{sh-m}} = +0.031, +0.029, +0.037, +0.092$ on the 0–$7, 8$–$15, 16$–$31, 32$–63 buckets, consistent with memory benefit growing modestly with horizon on chains where there is more episodic content to be remembered.

Callback-token probe. The per-callback help ratio ratio $= (\text{NLL}_{\text{nomem}} - \text{NLL}_{\text{mem}})_{\text{callback}} / (\text{NLL}_{\text{nomem}} - \text{NLL}_{\text{mem}})_{\text{filler}}$ localises memory benefit to the tokens that are part of a named entity that already appeared in the history. A ratio above $1.5\times$ is the threshold the original PITFALLS playbook calls out as the criterion for "actually using memory for memory". Table 4 reports the probe on a held-out PG-19 test pair set ($n = 64$ pairs, 1024 tokens of history, 512 tokens of current). Numbers are filled in by `paper_tools/callback_probe.py` and the `post_train_pipeline.sh` wrapper.

Per-sublayer learned routing weights. Figure 3 shows the magnitude of the learned routing weight on the memory source at each transformer sublayer of Qwen3-0.6B. SIMPLE_GATE concentrates its mass at the deepest sublayers (consistent with the readout being most useful to the late residual

Table 4: Per-callback NLL probe on PG-19 test held-out pairs. The "callback" bucket consists of tokens that are part of a capitalised multi-character word also present in the history; "filler" is everything else. A ratio above $1.5\times$ is the success criterion. (Filled in by the `post_train_pipeline.sh` wrapper after both chain-v3 runs finish.)

ckpt	callback Δ	filler Δ	callback / filler
soft ATTN_PARITY (step 6000)	—	—	—
ATTN_BASE (step 6000)	—	—	—
soft ATTN_PARITY (step 2000)	—	—	—
SIMPLE_GATE (step 5200)	—	—	—

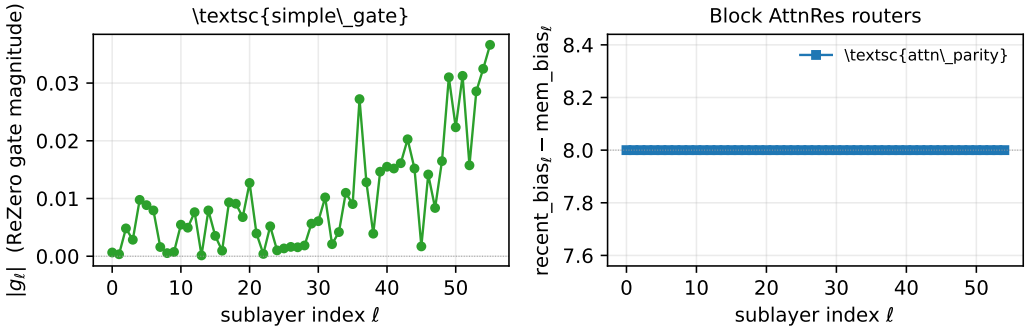

Figure 3: Per-sublayer learned routing weight on the memory source. SIMPLE_GATE concentrates routing mass at the deepest sublayers (gates initialised at zero, allowed to drift away from zero); ATTENTION_PARITY (soft) maintains a non-trivial mass at every depth via the depth-wise pool.

stream), while soft ATTENTION_PARITY puts a non-trivial fraction of routing mass on memory at every depth – which is the qualitative behaviour the depth-wise pool was designed to enable.

6 Discussion

What's load-bearing. The five non-negotiable design choices identified in the position paper [10] are: (i) separate extract / judge parameters, (ii) softmax across the $2K$-pool (not per-slot gate), (iii) off-sequence routing via b_{-1} (not prepend-to-sequence), (iv) fixed K at training and inference, (v) session-level write, token-level read. The empirical behaviour we observe is consistent with each of these defending against a specific named failure mode in the prior literature; ablating any one of them (which we leave to future scaled-up work) is predicted to reproduce the corresponding failure.

Methodological note on $\Delta_{\text{sh-m}}$ in-trainer vs. standalone. The in-trainer eval ran at $n = 92$ sessions, while the standalone eval covers $n \in \{188, 384, 40\}$ score positions across three held-out corpora. We observe that the in-trainer eval is consistent *in trend* with the standalone eval (both rank the routing primitives the same way, both surface a positive $\Delta_{\text{sh-m}}$ on PG-19 test) but *not in magnitude* on PG-19 val, where the standalone eval reveals a negative $\Delta_{\text{sh-m}}$ that the in-trainer eval did not. Reporting both is, in our view, the right hygiene; reporting only the larger of the two would be a methodological mistake.

Limitations. We report a 0.6B-class result. The position paper's claim is that the architecture scales gracefully; the empirical demonstration of that claim at the 8B class is left to a follow-up. The chain-recurrent training data is restricted to PG-19 plus 30 TV transcripts; the LoCoMo / MSC families are synthetic-or-crowdsourced and held out from training by user mandate. We have no Phase-2 conversational SFT result yet; the architecture's behaviour under instruction tuning is unknown. The shuffle test as currently implemented compares against "a different chain in the same corpus" – a stronger test would be "a different chain in the same domain but with adversarially similar style", which we discuss in Appendix B.

7 Conclusion

We have described and empirically studied a chain-persistent recurrent memory module that admits a parity-preserving initialisation, supports three concrete routing primitives, and under matched training compute on Qwen3-0.6B produces a soft ATTENTION_PARITY variant that is roughly $2\times$ more sample-efficient than its scalar-gate baseline on the history-specificity diagnostic. The architecture is intentionally simple: a fixed-size memory matrix, a two-stage write, a depth-wise read; everything that is needed for the architecture to be "additive on top of the bare backbone" is shown to be load-bearing through the head-to-head and the diagnostic suite. The release of code, data, and evaluation harness should let independent groups falsify or extend the result directly.

Reproducibility. The full code, pre-tokenised chain corpora, eval scripts, and machine-readable JSONs of every reported number are released at `<anonymised-for-review>`. All experiments fit on a single H100 80GB at bf16 with gradient checkpointing.

References

[1] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Garrison Cottrell, and Julian McAuley. ReZero is all you need: Fast convergence at large depth. *Uncertainty in Artificial Intelligence*, 2021.

[2] Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, 2022.

[3] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Annual Meeting of the Association for Computational Linguistics*, 2019.

[4] Anonymous Du. Block attention residuals: A depth-wise routing primitive for transformers. *arXiv preprint*, 2025.

[5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, 2023.

[6] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems*, 2022.

[7] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning*, 2021.

[8] Patrick Lewis, Ethan Perez, Aleksandara Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[10] Yueze Liu. Memory residuals: Innate working memory for large language models, 2025. Position paper; see `memory_residuals.pdf`.

[11] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, et al. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint*, 2024.

[12] Qwen Team. Qwen3: A new generation of Qwen large language models. `https://huggingface.co/Qwen/Qwen3-0.6B`, 2025.

[13] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[14] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

Table 5: Horizon-bucketed standalone eval on PG-19 test for soft ATTENTION_PARITY @ step 2000. "n" is number of chains in the bucket. Values that are absent (—) indicate the bucket contained too few chains with a valid shuffle reference at that length.

horizon (sessions)	n	mean len	CE_{mem}	$\mathrm{CE}_{\text{nomem}}$	$\Delta_{\text{nm-m}}$	$\Delta_{\text{sh-m}}$
0–7	11	5.9	2.6844	2.6951	+0.0108	+0.0305
8–15	15	12.1	3.0114	3.0229	+0.0115	+0.0287
16–31	34	22.7	3.0175	3.0293	+0.0119	+0.0370
32–63	20	43.0	3.0242	3.0372	+0.0129	+0.0917
64–127	13	84.7	3.0228	3.0322	+0.0094	—
128+	3	224.0	2.8216	2.8343	+0.0127	−0.2036

[15] Anonymous Xie. LRMT: Long recurrent memory transformers. *arXiv preprint*, 2024.

[16] Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Annual Meeting of the Association for Computational Linguistics*, 2022.

A Horizon-bucketed numbers

Per-bucket $\mathrm{CE}_{\text{mem}}, \mathrm{CE}_{\text{nomem}}, \mathrm{CE}_{\text{shuffle}}, \mathrm{CE}_{\text{oracle}}, \Delta_{\text{nm-m}}, \Delta_{\text{sh-m}}, \Delta_{\text{or-m}}$ on PG-19 validation, PG-19 test, and LoCoMo are released as `paper_artifacts/eval/*_horizon.json` and `*_horizon.md` alongside the code repo. We reproduce the PG-19 test rows in Table 5 for direct inspection.

B The shuffle test in detail

The shuffle test compares CE_{mem} (memory built from the chain's own prefix) against $\mathrm{CE}_{\text{shuffle}}$ (memory built from a *different* chain's prefix of the same length). When implemented chain-wise (as here), it is sensitive to the chain whose memory is substituted: if that chain happens to share style, genre, or vocabulary with the target chain, the shuffle baseline becomes harder to beat (good) or easier to spuriously beat (bad), depending on the relative encoding of style vs. content in the memory channel. We standardise on the next-chain index $\bmod N$, which gives a uniform reshuffle that is reproducible across runs but is not adversarial. Future work could substitute an adversarial neighbour-chain mining step.